Moderando el extremismo de opiniones en una red social (ModEx)

Proyecto de curso – Análisis y Diseño de Algoritmos II
Escuela de Ingeniería de Sistemas y Computación

Santiago Arias Rojas (2416285) Sebastian Calvo Carvajal (2419918)
Sebastian Rubio Piedrahita (2459628)

Profesor: Jesús Alexander Aranda Monitor: Samuel Galindo

Noviembre de 2026

Índice

1. Introducción **3**

2. El problema **3**
2.1. Formalización . . . 3
2.2. Reformulación: una mochila 0/1 . . . 3
2.3. Ejemplos del enunciado . . . 4

3. Fuerza bruta **4**
3.1. Complejidad . . . 5
3.2. Corrección . . . 5

4. Algoritmo voraz **5**
4.1. Descripción del algoritmo . . . 5
4.2. Entendimos el algoritmo . . . 6
4.3. Complejidad . . . 7
4.4. Corrección . . . 7

5. Programación dinámica **8**
5.1. Caracterización de la estructura de una solución óptima . . . 8
5.2. Definición recursiva del valor de una solución óptima . . . 9
5.3. Algoritmo para calcular el costo de una solución óptima . . . 9
5.4. Algoritmo para calcular una solución óptima . . . 9
5.5. Complejidad . . . 11

6. Implementación **12**
6.1. Tecnología y estructura . . . 12
6.2. Interfaz . . . 12
6.3. Formatos de entrada y salida . . . 13

7. Comparación de las tres soluciones **13**
7.1. Diseño de los casos de prueba 13
7.2. Resultados en los casos diseñados 13
7.3. Calidad del voraz en casos aleatorios 14
7.4. Tiempos de ejecución 14

8. Conclusiones **15**

1. Introducción

En este informe se estudia el problema ModEx: dada una red social de n agentes, cada uno con una opinión, una receptividad y una influencia, y un valor máximo $R_{\text{máx}}$ disponible para moderar opiniones, se debe escoger a qué agentes moderar (llevar su opinión a 0) para que el extremismo de la red resultante sea mínimo sin superar $R_{\text{máx}}$.

Se diseñaron, analizaron e implementaron tres soluciones: fuerza bruta (`modexFB`), un algoritmo voraz (`modexV`) y programación dinámica (`modexPD`). Para cada una se estudia su complejidad y su corrección, y al final se comparan experimentalmente y se concluye sobre sus ventajas y desventajas en la práctica.

2. El problema

2.1. Formalización

Una red social es una pareja $RS = \langle AG, R_{\text{máx}} \rangle$ con $AG = \langle a_0, \ldots, a_{n-1} \rangle$ y $R_{\text{máx}} \in \mathbb{N}$. Cada agente es una terna $a_i = \langle o_i, r_i, inf_i \rangle$ con $o_i \in \{-100, \ldots, 100\}$, $r_i \in [0, 1]$ e $inf_i \in [1, 3]$.

El extremismo de la red es

$$\text{Ext}(RS) = \frac{\sqrt{\sum_{i=0}^{n-1} o_i^2 \cdot inf_i}}{n}.$$

Esta es la lectura de la fórmula que reproduce los valores de los ejemplos del enunciado (47.14, 33.5 y 9.43), y es la que usa la implementación.

Una estrategia es $E = \langle e_0, \ldots, e_{n-1} \rangle$ con $e_i \in \{0, 1\}$; $\text{Mod}(RS, E)$ cambia a 0 la opinión de los agentes con $e_i = 1$. El esfuerzo es

$$\text{Esfuerzo}(RS, E) = \sum_{i=0}^{n-1} \lceil |o_i - o_i'| \cdot (1 - r_i) \cdot inf_i \rceil .$$

E es aplicable si $\text{Esfuerzo}(RS, E) \leq R_{\text{máx}}$. ModEx pide una estrategia aplicable que minimice $\text{Ext}(\text{Mod}(RS, E))$.

2.2. Reformulación: una mochila 0/1

Para cada agente definimos su *costo* y su *beneficio*:

$$c_i = \lceil |o_i| \cdot (1 - r_i) \cdot inf_i \rceil , \qquad b_i = o_i^2 \cdot inf_i.$$

Si $e_i = 0$ entonces $o_i' = o_i$ y el término del esfuerzo es $\lceil 0 \rceil = 0$; si $e_i = 1$ entonces $o_i' = 0$ y el término es c_i. Por lo tanto

$$\text{Esfuerzo}(RS, E) = \sum_i e_i \, c_i, \qquad \text{Ext}(\text{Mod}(RS, E)) = \frac{\sqrt{S - B(E)}}{n}, \quad \text{con } S = \sum_i b_i, \; B(E) = \sum_i e_i \, b_i.$$

Como $x \mapsto \sqrt{S - x}/n$ es decreciente, **minimizar el extremismo equivale a maximizar** $B(E)$ **sujeto a** $\sum_i e_i c_i \leq R_{\text{máx}}$: es exactamente el problema de la mochila 0/1, donde los agentes son los objetos, c_i su peso, b_i su valor y $R_{\text{máx}}$ la capacidad. Esta observación guía el diseño del algoritmo voraz y del de programación dinámica.

Dos cotas útiles: como $|o_i| \leq 100$, $1 - r_i \leq 1$ e $inf_i \leq 3$, se tiene $0 \leq c_i \leq 300$ y por lo tanto $\sum_i c_i \leq 300n$.

Además, la estrategia $E = \langle 0, \ldots, 0 \rangle$ tiene esfuerzo $0 \leq R_{\text{máx}}$, así que **siempre existe al menos una estrategia aplicable**.

2.3. Ejemplos del enunciado

Red	Agente	$\langle o, r, inf \rangle$	c_i	b_i
RS_1, $R_{\text{máx}} = 55$	a_0	$\langle 100, 0.5, 1 \rangle$	50	10000
	a_1	$\langle 100, 0.1, 1 \rangle$	90	10000
	a_2	$\langle -10, 0.1, 1 \rangle$	9	100
RS_2, $R_{\text{máx}} = 35$	a_0	$\langle -30, 0.9, 1 \rangle$	3	900
	a_1	$\langle 20, 0.1, 2 \rangle$	36	800
	a_2	$\langle 50, 0.5, 1 \rangle$	25	2500

Tabla 1: Costos y beneficios de los agentes de los ejemplos 2.3.1 y 2.3.2.

En RS_1 las estrategias aplicables no vacías son $\langle 1, 0, 0 \rangle$ (Ext = 33.50) y $\langle 0, 0, 1 \rangle$ (Ext = 47.14); $\langle 1, 0, 1 \rangle$ cuesta $59 > 55$ y cualquier estrategia que modere a a_1 cuesta al menos 90. El óptimo es $\langle 1, 0, 0 \rangle$ con esfuerzo 50 y extremismo 33.50.

En RS_2 el óptimo es $\langle 1, 0, 1 \rangle$ con esfuerzo 28 y extremismo $\sqrt{800}/3 = 9.43$: moderar a a_1 cuesta $36 > 35$, así que solo se pueden moderar subconjuntos de $\{a_0, a_2\}$ y moderar a ambos es lo mejor.

3. Fuerza bruta

El algoritmo genera todas las estrategias posibles y entre ellas escoge la mejor aplicable. Las estrategias se generan recursivamente fijando $e_i = 0$ y luego $e_i = 1$ para cada agente; cada estrategia completa se evalúa apenas se genera, de modo que no es necesario guardar las 2^n estrategias en memoria.

Algoritmo 1: Fuerza bruta (`modexFB`)

Input: Red $RS = \langle AG, R_{\text{máx}} \rangle$ con n agentes
Output: $(E, \text{Esfuerzo}(RS, E), \text{Ext}(\text{Mod}(RS, E)))$ óptima

```
1  E ← ⟨0, ..., 0⟩; mejor ← (∅, 0, +∞);
2  Explorar(0);
3  return mejor;
4  Función Explorar(i):
5      if i = n then
6          esf ← Esfuerzo(RS, E);                                   // O(n)
7          if esf ≤ R_máx then
8              ext ← Ext(Mod(RS, E));                               // O(n)
9              if ext < mejor.ext o (ext = mejor.ext y esf < mejor.esf) then
10                 mejor ← (E, esf, ext);
11     else
12         e_i ← 0; Explorar(i + 1);
13         e_i ← 1; Explorar(i + 1);
14         e_i ← 0;
```

3.1. Complejidad

El árbol de recursión es un árbol binario completo de altura n: tiene 2^n hojas (una por estrategia) y $2^n - 1$ nodos internos. Cada nodo interno hace trabajo constante y cada hoja calcula el esfuerzo y el extremismo en $O(n)$. Equivalentemente, $T(n)$ cumple $T(i) = 2T(i+1) + O(1)$ para $i < n$ y $T(n) = O(n)$ en las hojas, de donde

$$T(n) = (2^n - 1) \cdot O(1) + 2^n \cdot O(n) = \Theta(n \cdot 2^n).$$

En espacio, la profundidad de la recursión es n y solo se guardan la estrategia actual y la mejor, por lo que el espacio es $\Theta(n)$.

Una primera versión del código construía la lista completa de las 2^n estrategias antes de evaluarlas, lo que requería $\Theta(n \cdot 2^n)$ de memoria (unos 190 MB para $n = 20$). Se reemplazó por la exploración recursiva descrita, que tiene el mismo orden de tiempo y espacio lineal.

3.2. Corrección

Sí, el algoritmo siempre da la respuesta correcta. Toda estrategia de moderación es una secuencia de n bits, y la recursión visita cada una de las 2^n secuencias exactamente una vez (en cada nivel i se toman las dos opciones de e_i). Entre ellas descarta solo las no aplicables y conserva la de menor extremismo. Como $\langle 0, \ldots, 0 \rangle$ siempre es aplicable, el conjunto de candidatas no es vacío y el resultado es una estrategia aplicable de extremismo mínimo. En caso de empate en extremismo se prefiere la de menor esfuerzo, lo que no afecta la optimalidad.

Detalle de implementación: el redondeo. El esfuerzo se calcula con números de punto flotante, donde por ejemplo $1 - 0.7 = 0.30000000000000004$. Para el agente $\langle 10, 0.7, 1 \rangle$ el valor exacto es $\lceil 10 \cdot 0.3 \cdot 1 \rceil = 3$, pero aplicar `ceil` directamente da 4, con lo que una estrategia aplicable (por ejemplo con $R_{\text{máx}} = 3$) se descartaría. La función `techo` resta una tolerancia de 10^{-9} antes de aplicar `ceil`, muy inferior a la precisión de los datos de entrada. Este caso está incluido en las pruebas (caso E_7 de la tabla 3).

4. Algoritmo voraz

4.1. Descripción del algoritmo

Por la reformulación de la sección 2.2, cada agente aporta un beneficio b_i (reducción de S) a cambio de un costo c_i. La idea voraz es moderar primero a los agentes que más reducen el extremismo *por unidad de esfuerzo*, es decir, los de mayor razón b_i/c_i:

1. Descartar los agentes con $b_i = 0$ (opinión 0, no aportan) y los que cuestan más que $R_{\text{máx}}$ (nunca pueden moderarse).

2. Ordenar los agentes restantes por b_i/c_i de mayor a menor. Los de costo 0 (receptividad 1) tienen razón $+\infty$ y van primero; los empates se rompen por mayor beneficio.

3. Recorrer la lista en ese orden y moderar a cada agente cuyo costo aún quepa en el presupuesto restante. El recorrido no se detiene en el primer agente que no cabe: sigue buscando agentes más baratos.

4. Repetir el paso 3 empezando con el agente de mayor beneficio $b_{\text{máx}}$ ya moderado, y quedarse con la mejor de las dos soluciones.

El paso 4 corrige el caso en que la razón favorece a agentes baratos pero poco extremos que consumen el presupuesto que necesitaba un agente muy extremo (caso E_4 de la tabla 3). Cuesta solo un recorrido adicional y, como se demuestra en la sección 4.4, da una garantía de calidad que el paso 3 solo no tiene.

Algoritmo 2: Algoritmo voraz (`modexV`)

Input: Red $RS = \langle AG, R_{\text{máx}} \rangle$ con n agentes
Output: $(E, \text{Esfuerzo}(RS, E), \text{Ext}(\text{Mod}(RS, E)))$

$L \leftarrow \{\, i : b_i > 0$ y $c_i \leq R_{\text{máx}} \,\}$;
ordenar L por b_i/c_i descendente (razón $+\infty$ si $c_i = 0$);
$E \leftarrow \langle 0, \ldots, 0 \rangle$; $(E, B) \leftarrow$ `Llenar(`E, 0, 0`)`;
$m \leftarrow \arg\max_{i \in L} b_i$;
if $L \neq \emptyset$ ***y*** $e_m = 0$ **then**
    $E' \leftarrow \langle 0, \ldots, 0 \rangle$; $e'_m \leftarrow 1$;
    $(E', B') \leftarrow$ `Llenar(`E', c_m, b_m`)`;
    if $B' > B$ **then** $E \leftarrow E'$;
return $(E, \text{Esfuerzo}(RS, E), \text{Ext}(\text{Mod}(RS, E)))$;
Función `Llenar(`E, *usado*, B`)`**:**
    for $i \in L$ *en orden* **do**
        if $e_i = 0$ ***y*** $usado + c_i \leq R_{\text{máx}}$ **then**
            $e_i \leftarrow 1$; $usado \leftarrow usado + c_i$; $B \leftarrow B + b_i$;
    return (E, B);

4.2. Entendimos el algoritmo

Ejemplo 2.3.1 (RS_1, $R_{\text{máx}} = 55$). Las razones son $a_0 : 10000/50 = 200$, $a_1 : 10000/90 = 111.1$ y $a_2 : 100/9 = 11.1$, pero a_1 se descarta porque $c_1 = 90 > 55$. El orden es a_0, a_2. Se modera a a_0 (usado $= 50$); a_2 no cabe ($59 > 55$). En el paso 4, el agente de mayor beneficio es a_0, que ya está moderado. La salida es $E = \langle 1, 0, 0 \rangle$, con esfuerzo 50 y extremismo $\sqrt{10100}/3 = 33.50$. **Es la solución óptima.**

Ejemplo 2.3.2 (RS_2, $R_{\text{máx}} = 35$). Se descarta a_1 ($c_1 = 36 > 35$). Las razones son $a_0 : 900/3 = 300$ y $a_2 : 2500/25 = 100$. Se modera a a_0 (usado $= 3$) y luego a a_2 (usado $= 28$). En el paso 4, el de mayor beneficio es a_2, ya moderado. La salida es $E = \langle 1, 0, 1 \rangle$, con esfuerzo 28 y extremismo $\sqrt{800}/3 = 9.43$. **Es la solución óptima.**

Otras entradas. Se diseñaron seis entradas adicionales, cada una para ejercitar una situación distinta (tabla 2). La tabla 3 compara la salida del voraz con el óptimo, calculado con fuerza bruta y confirmado con programación dinámica.

Comentarios sobre los casos:

- E_3: costos $6, 5, 5$; beneficios $3600, 2500, 2500$; razones $600, 500, 500$. El voraz modera a a_0 y le sobran 4 unidades en las que no cabe nadie. El paso 4 empieza también por a_0 (mayor beneficio). El óptimo modera a a_1 y a_2 usando todo el presupuesto: $B = 5000 > 3600$.

Caso	$R_{\text{máx}}$	Agentes $\langle o, r, inf \rangle$	Propósito
E_1	55	$\langle 100, 0.5, 1\rangle, \langle 100, 0.1, 1\rangle, \langle -10, 0.1, 1\rangle$	Ejemplo 2.3.1
E_2	35	$\langle -30, 0.9, 1\rangle, \langle 20, 0.1, 2\rangle, \langle 50, 0.5, 1\rangle$	Ejemplo 2.3.2
E_3	10	$\langle 60, 0.9, 1\rangle, \langle 50, 0.9, 1\rangle, \langle 50, 0.9, 1\rangle$	Contraejemplo del voraz
E_4	50	$\langle 20, 0.95, 1\rangle, \langle 100, 0.5, 1\rangle$	Necesidad del paso 4
E_5	0	$\langle 80, 1, 2\rangle, \langle -50, 0.2, 1.5\rangle, \langle 0, 0.3, 3\rangle, \langle 30, 0.6, 1\rangle$	$R_{\text{máx}} = 0$, costo 0 y opinión 0
E_6	500	$\langle -100, 0.2, 3\rangle, \langle 80, 0.6, 2\rangle, \langle -20, 0, 1.2\rangle$	Alcanza para todos
E_7	3	$\langle 10, 0.7, 1\rangle, \langle -50, 0.5, 1\rangle$	Redondeo de punto flotante
E_8	90	$\langle 90, 0.5, 1\rangle, \langle -60, 0.5, 1\rangle, \langle -60, 0.5, 1\rangle, \langle 40, 0, 1\rangle, \langle -70, 0.4, 2\rangle$	Óptimos empatados

Tabla 2: Entradas diseñadas para analizar el algoritmo voraz.

	Voraz			Óptimo (FB = PD)			
Caso	E	Esf	Ext	E	Esf	Ext	¿Óptima?
E_1	$\langle 1, 0, 0\rangle$	50	33.4996	$\langle 1, 0, 0\rangle$	50	33.4996	Sí
E_2	$\langle 1, 0, 1\rangle$	28	9.4281	$\langle 1, 0, 1\rangle$	28	9.4281	Sí
E_3	$\langle 1, 0, 0\rangle$	6	23.5702	$\langle 0, 1, 1\rangle$	10	20.0000	**No**
E_4	$\langle 0, 1\rangle$	50	10.0000	$\langle 0, 1\rangle$	50	10.0000	Sí
E_5	$\langle 1, 0, 0, 0\rangle$	0	17.0477	$\langle 1, 0, 0, 0\rangle$	0	17.0477	Sí
E_6	$\langle 1, 1, 1\rangle$	328	0.0000	$\langle 1, 1, 1\rangle$	328	0.0000	Sí
E_7	$\langle 1, 0\rangle$	3	25.0000	$\langle 1, 0\rangle$	3	25.0000	Sí
E_8	$\langle 1, 1, 0, 0, 0\rangle$	75	24.4949	$\langle 1, 0, 1, 0, 0\rangle$	75	24.4949	Sí (empate)

Tabla 3: Salida del algoritmo voraz frente a la solución óptima.

- E_4: a_0 tiene costo 1 y razón 400; a_1 tiene costo 50 y razón 200. El paso 3 modera a a_0 y ya no cabe a_1 ($B = 400$). El paso 4 empieza por a_1 ($B = 10000$) y obtiene el óptimo. Sin el paso 4 el extremismo sería 50 en lugar de 10.
- E_5: con $R_{\text{máx}} = 0$ solo se puede moderar al agente de receptividad 1 (costo 0). El agente de opinión 0 no se modera porque no aporta beneficio.
- E_7: sin la tolerancia de `techo` el costo de a_0 sería $4 > 3$ y ningún algoritmo lo moderaría.
- E_8: $\{a_0, a_1\}$ y $\{a_0, a_2\}$ son óptimos (los dos agentes de opinión -60 son idénticos); los algoritmos devuelven estrategias distintas con el mismo esfuerzo y extremismo.

4.3. Complejidad

Calcular c_i y b_i y filtrar es $O(n)$. Ordenar $|L| \leq n$ agentes es $O(n \log n)$. Cada llamada a `Llenar` recorre la lista una vez, $O(n)$, y se hacen a lo sumo dos. Encontrar $b_{\text{máx}}$ y calcular el esfuerzo y el extremismo finales es $O(n)$. En total

$$T(n) = O(n) + O(n \log n) + 2 \cdot O(n) = \Theta(n \log n),$$

dominado por el ordenamiento. El espacio es $\Theta(n)$ (la lista ordenada y dos estrategias).

4.4. Corrección

No, el algoritmo voraz no siempre da la respuesta correcta. El caso E_3 es un contraejemplo verificado: el voraz entrega extremismo 23.57 y el óptimo es 20. El motivo es que ModEx

es una mochila 0/1, que no tiene la propiedad de elección voraz: escoger al agente de mejor razón puede dejar presupuesto sin usar que ningún otro agente alcanza a aprovechar. Ordenar por razón solo es óptimo en la versión fraccionaria, donde se podría moderar una fracción de un agente.

Cuándo sí da la respuesta correcta.

1. Si alcanza para todos ($\sum_i c_i \leq R_{\text{máx}}$): modera a todos los agentes con $b_i > 0$, lo que da extremismo 0 (caso E_6).
2. Si $R_{\text{máx}} = 0$: solo pueden moderarse agentes de costo 0, y los modera a todos (caso E_5).
3. Si todos los costos son iguales: el orden por razón coincide con el orden por beneficio y modera a los $\lfloor R_{\text{máx}}/c \rfloor$ de mayor beneficio, que es óptimo.
4. Si todos los beneficios son iguales: el orden por razón es el de menor costo, que maximiza la cantidad de agentes moderados, lo que es óptimo.
5. Si los agentes moderados en el paso 3 son exactamente los primeros de la lista ordenada y su costo suma exactamente $R_{\text{máx}}$: la solución alcanza la cota de la mochila fraccionaria y nada puede superarla.

Cuándo no. Falla cuando, tras moderar a los agentes de mejor razón, queda presupuesto "desperdiciado" que una combinación distinta, de razón individual menor, sí aprovecharía (caso E_3). Esto ocurre con más frecuencia cuando hay pocos agentes con costos grandes respecto a $R_{\text{máx}}$.

Proposición 1. *Sea B^* el beneficio óptimo y B_V el beneficio de la solución del voraz. Entonces $B_V \geq B^*/2$.*

Demostración. Sean $u_1, \ldots, u_m$ los agentes de L en el orden por razón. Si todos caben, el paso 3 los modera a todos y es óptimo. Si no, sea u_k el primero que no cabe en el paso 3; los anteriores $u_1, \ldots, u_{k-1}$ sí fueron moderados, así que $B_3 \geq \sum_{l<k} b_{u_l}$. El óptimo de la mochila fraccionaria toma $u_1, \ldots, u_{k-1}$ y una fracción de u_k, luego $B^* \leq \sum_{l<k} b_{u_l} + b_{u_k} \leq B_3 + b_{\text{máx}}$. El paso 4 parte del agente de beneficio $b_{\text{máx}}$, así que $B_4 \geq b_{\text{máx}}$ (si ese agente ya fue moderado en el paso 3, entonces $B_3 \geq b_{\text{máx}}$). Por lo tanto $B_V = \text{máx}(B_3, B_4) \geq (B_3 + b_{\text{máx}})/2 \geq B^*/2$. □

Sin el paso 4 no hay garantía: con un agente de costo 1 y beneficio 2 y otro de costo $R_{\text{máx}}$ y beneficio $R_{\text{máx}}$, el paso 3 solo obtendría 2 frente a un óptimo de $R_{\text{máx}}$, arbitrariamente peor (el caso E_4 es una instancia de esta situación). La garantía es sobre la reducción B, no sobre el extremismo: si el óptimo deja la red casi sin extremismo, el error relativo en Ext puede ser grande (en las pruebas aleatorias llegó al 206 % en un caso donde el óptimo era cercano a 0).

La tabla 7 resume el comportamiento observado en 3600 redes aleatorias: el voraz entrega el óptimo exacto en la mayoría de los casos pequeños, y aunque la proporción de óptimos exactos baja al crecer n, el error relativo promedio en el extremismo cae a menos de 0.01 % para $n = 1000$, porque con muchos agentes el presupuesto desperdiciado es pequeño frente al total.

5. Programación dinámica

5.1. Caracterización de la estructura de una solución óptima

Consideremos el subproblema $P(i, j)$: *escoger qué agentes moderar entre los primeros i agentes $a_0, \ldots, a_{i-1}$ con un esfuerzo de a lo sumo j, maximizando el beneficio moderado.* El problema original es $P(n, R_{\text{máx}})$.

Lema 1 (Subestructura óptima)**.** *Sea $E^* = \langle e_0, \ldots, e_{i-1} \rangle$ una solución óptima de $P(i, j)$ con $i \geq 1$.*

- *Si $e_{i-1} = 0$, entonces $\langle e_0, \ldots, e_{i-2} \rangle$ es una solución óptima de $P(i-1, j)$.*
- *Si $e_{i-1} = 1$, entonces $c_{i-1} \leq j$ y $\langle e_0, \ldots, e_{i-2} \rangle$ es una solución óptima de $P(i-1, j-c_{i-1})$.*

Demostración. Caso $e_{i-1} = 0$: el prefijo usa esfuerzo $\leq j$ y solo agentes entre $a_0, \ldots, a_{i-2}$, luego es solución de $P(i-1, j)$. Si existiera una solución de $P(i-1, j)$ con mayor beneficio, al agregarle $e_{i-1} = 0$ se obtendría una solución de $P(i, j)$ mejor que E^*, una contradicción.

Caso $e_{i-1} = 1$: como el esfuerzo total es $\leq j$ y a_{i-1} aporta c_{i-1}, el prefijo usa a lo sumo $j - c_{i-1}$. Si existiera una solución de $P(i-1, j-c_{i-1})$ con mayor beneficio, al agregarle $e_{i-1} = 1$ se tendría una solución de $P(i, j)$ con esfuerzo $\leq j$ y beneficio mayor que el de E^*, una contradicción. □

Así, una solución óptima de $P(i, j)$ se construye a partir de una solución óptima de $P(i-1, j)$ o de $P(i-1, j-c_{i-1})$. Los subproblemas necesarios son

$$\{\, P(i, j) : 0 \leq i \leq n,\ 0 \leq j \leq A \,\}, \qquad A = \min\left(R_{\text{máx}},\ \sum_i c_i\right).$$

Basta llegar hasta A en lugar de $R_{\text{máx}}$: con un presupuesto mayor que el costo de moderar a todos, el presupuesto deja de ser una restricción. Hay $(n+1)(A+1)$ subproblemas, y como $A \leq 300n$, son $O(n^2)$.

5.2. Definición recursiva del valor de una solución óptima

Sea $M(i, j)$ el máximo beneficio de $P(i, j)$:

$$M(i,j) = \begin{cases} 0 & \text{si } i = 0, \\ M(i-1, j) & \text{si } i > 0 \text{ y } c_{i-1} > j, \\ \max\big(M(i-1, j),\ M(i-1, j-c_{i-1}) + b_{i-1}\big) & \text{si } i > 0 \text{ y } c_{i-1} \leq j. \end{cases}$$

El costo (extremismo) de una solución óptima del problema original es

$$\text{Ext}^* = \frac{\sqrt{S - M(n, A)}}{n}, \qquad S = \sum_i b_i.$$

De manera equivalente, el extremismo restante mínimo $X(i, j) = \sum_{l<i} b_l - M(i, j)$ satisface $X(0, j) = 0$, $X(i, j) = X(i-1, j) + b_{i-1}$ si $c_{i-1} > j$ y $X(i, j) = \min\big(X(i-1, j) + b_{i-1},\ X(i-1, j-c_{i-1})\big)$ en otro caso, con $\text{Ext}^* = \sqrt{X(n, A)}/n$.

5.3. Algoritmo para calcular el costo de una solución óptima

La fila i de M solo depende de la fila $i-1$, así que la tabla se llena de abajo hacia arriba, fila por fila:

5.4. Algoritmo para calcular una solución óptima

Se agrega una tabla de decisiones $D[i][j] \in \{0, 1\}$ con $D[i][j] = 1$ si y solo si el valor $M[i][j]$ se obtuvo moderando a a_{i-1} (es decir, si en el algoritmo 3 se ejecutó la línea de la condición). Luego se recorre la tabla desde (n, A) hacia atrás:

En cada paso, si $D[i][j] = 1$ el óptimo de $P(i, j)$ modera a a_{i-1} y continúa en $P(i-1, j-c_{i-1})$; si no, continúa en $P(i-1, j)$. Por el lema de subestructura óptima, la estrategia construida tiene beneficio $M(n, A)$ y es óptima. La reconstrucción toma $O(n)$.

Algoritmo 3: Costo óptimo por programación dinámica

Input: Red RS con n agentes
Output: Ext*

calcular c_i y b_i para $0 \le i < n$; $S \leftarrow \sum_i b_i$;
$A \leftarrow \min(R_{\text{máx}}, \sum_i c_i)$;
for $j \leftarrow 0$ **to** A **do** $M[0][j] \leftarrow 0$;
for $i \leftarrow 1$ **to** n **do**
  for $j \leftarrow 0$ **to** A **do**
    $M[i][j] \leftarrow M[i-1][j]$;
    if $c_{i-1} \le j$ ***y*** $M[i-1][j-c_{i-1}] + b_{i-1} > M[i][j]$ **then**
      $M[i][j] \leftarrow M[i-1][j-c_{i-1}] + b_{i-1}$;
return $\sqrt{S - M[n][A]}\,/\,n$;

Algoritmo 4: Reconstrucción de una estrategia óptima

Input: Tabla D, costos c_i, n, A
Output: Estrategia óptima E

$E \leftarrow \langle 0, \ldots, 0 \rangle$; $j \leftarrow A$;
for $i \leftarrow n$ ***bajando hasta*** 1 **do**
  if $D[i][j] = 1$ **then**
    $e_{i-1} \leftarrow 1$; $j \leftarrow j - c_{i-1}$;
return $(E, \text{Esfuerzo}(RS, E), \text{Ext}(\text{Mod}(RS, E)))$;

En la implementación. Como la fila i solo depende de la fila $i-1$, `modexPD` guarda los valores en un solo arreglo de $A+1$ reales que se recorre con j de A hacia 0 (así $M[j-c]$ todavía corresponde a la fila anterior), y la tabla D como una matriz de bits (`std::vector<bool>`). El núcleo del algoritmo es:

```
std::vector<double> M(A + 1, 0.0);
std::vector<std::vector<bool>> moderar(n, std::vector<bool>(A + 1, false));
for (std::size_t i = 0; i < n; ++i) {
    const auto c = static_cast<std::size_t>(costo[i]);
    if (c > A) continue;
    for (std::size_t j = A + 1; j-- > c;) {
        double conAgente = M[j - c] + beneficio[i];
        if (conAgente > M[j]) {
            M[j] = conAgente;
            moderar[i][j] = true;
        }
    }
}
std::vector<int> E(n, 0);
std::size_t j = A;
for (std::size_t i = n; i-- > 0;) {
    if (moderar[i][j]) {
        E[i] = 1;
        j -= static_cast<std::size_t>(costo[i]);
    }
```

```
21 }
```

5.5. Complejidad

Complejidad en tiempo. Calcular costos y beneficios es $O(n)$. Llenar la tabla son $(n+1)(A+1)$ celdas de costo constante, y la reconstrucción es $O(n)$. Por lo tanto $T(n) = \Theta(n \cdot A)$. Como $A \leq \sum_i c_i \leq 300n$,

$$T(n) = O(300\, n^2) = O(n^2).$$

En general el algoritmo es pseudopolinomial en $R_{\text{máx}}$, pero en este problema el presupuesto útil está acotado por $300n$, lo que lo hace polinomial en n.

Complejidad en espacio. Las tablas M y D tienen $(n+1)(A+1) \approx n \cdot A \leq 300n^2$ celdas, luego el espacio es $\Theta(n \cdot A) = O(n^2)$. (La implementación reduce M a una fila, pero D sigue siendo de $n \times (A+1)$, por lo que el orden se mantiene.)

¿Es útil en la práctica? Supongamos un equipo que procesa 3×10^8 operaciones por minuto y usa 4 bytes por celda, con $n = 2^k$. Tomamos el peor caso $A = 300n$ (con las aproximaciones $n+1 \sim n$, $A+1 \sim A$):

$$\text{operaciones} \approx n \cdot A = 300n^2 = 300 \cdot 4^k \;\Rightarrow\; \text{tiempo} \approx \frac{300 \cdot 4^k}{3 \times 10^8} = \frac{4^k}{10^6} \text{ minutos},$$

$$\text{espacio} \approx 4 \cdot 300n^2 = 1200 \cdot 2^{2k} \text{ bytes}.$$

Con 1 año = 518400 minutos y 2^{10} bytes = 1 KB, etc.:

k	$n = 2^k$	Tiempo	Espacio	Veredicto
5	32	0.001 min (0.06 s)	1.17 MB	Aceptable
8	256	0.066 min (3.9 s)	75 MB	Aceptable
10	1 024	1.05 min	1.17 GB	Aceptable
12	4 096	16.8 min	18.75 GB	Tiempo aceptable, espacio no
14	16 384	268 min (4.5 h)	300 GB	Inaceptable
16	65 536	4 295 min (3 días)	4.69 TB	Inaceptable
18	262 144	68 719 min (0.13 años)	75 TB	Inaceptable
20	1 048 576	1.1×10^6 min (2.1 años)	1.17 PB	Inaceptable

Tabla 4: Tiempo y espacio de la programación dinámica en el peor caso ($A = 300n$).

- **Tiempo:** es aceptable hasta $k \approx 12$ (unos 17 minutos); $k = 14$ ya toma más de 4 horas y para $k \geq 16$ toma días o años.
- **Espacio:** es aceptable hasta $k \approx 10$ (un poco más de 1 GB, que cabe en la memoria de un computador personal); para $k = 12$ se necesitan casi 19 GB y para $k \geq 14$ cientos de GB o más, lo que es inaceptable. El espacio es el factor que limita primero.

Este es el peor caso, con todos los agentes extremos, poco receptivos y muy influyentes, y un $R_{\text{máx}}$ que alcanza para moderarlos a todos. En las redes aleatorias de las pruebas, con $R_{\text{máx}}$ igual

a la mitad del esfuerzo total, se obtuvo $A \approx 25n$, doce veces menos. Además, la implementación guarda D con 1 bit por celda en lugar de 4 bytes, lo que divide la memoria entre 32: para $k = 12$ en el peor caso usaría unos 600 MB. El equipo de prueba es también mucho más rápido que el modelo del enunciado: llena unas 8×10^8 celdas por segundo (tabla 8).

6. Implementación

6.1. Tecnología y estructura

El programa está escrito en C++17 con Qt 6 (Widgets) para la interfaz y CMake para la construcción. La lógica está en una librería (`modex_core`) que usan tanto la aplicación gráfica (`ModEx`) como un programa de pruebas en consola (`pruebas`).

Archivo	Contenido
`include/agente.hpp`, `src/agente.cpp`	Clase `Agente` y generación aleatoria
`include/red.hpp`, `src/red.cpp`	Clase `Red`, Ext, costo, esfuerzo, archivos
`include/resultado.hpp`	Tripleta $(E, \text{Esfuerzo}, \text{Ext})$
`src/estrategias/fuerzabruta.cpp`	`modexFB`
`src/estrategias/voraz.cpp`	`modexV`
`src/estrategias/dinamica.cpp`	`modexPD`
`src/mainwindow.cpp`	Interfaz gráfica
`pruebas/pruebas.cpp`	Pruebas automáticas y mediciones
`casos/`	Ejemplos del enunciado en el formato de entrada

Tabla 5: Archivos principales del proyecto.

Las tres funciones pedidas, `modexFB`, `modexV` y `modexPD`, reciben una red (`const Red&`) y devuelven un `Resultado` con la estrategia E, su esfuerzo y el extremismo de la red moderada.

6.2. Interfaz

La interfaz permite:

- **Abrir** un archivo de entrada y visualizar la red: una tabla con la opinión, la receptividad, la influencia y el costo de moderar a cada agente, además de n, $R_{\text{máx}}$, el esfuerzo de moderar a todos y el extremismo inicial.
- **Generar casos de prueba** aleatorios con n y $R_{\text{máx}}$ escogidos por el usuario; el caso se guarda en un archivo con el formato de entrada y se carga.
- Ejecutar **Fuerza bruta**, **Voraz** o **Dinámica**, o **Comparar** las tres (la fuerza bruta solo se habilita para $n \leq 25$). Los algoritmos corren en un hilo aparte para no congelar la ventana. La estrategia de cada algoritmo aparece en su propia columna de la tabla de agentes.
- **Revisar la calidad de las salidas**: una tabla de resultados muestra para cada algoritmo Ext, Esfuerzo, si Esfuerzo $\leq R_{\text{máx}}$, si el recálculo de Esfuerzo y Ext a partir de E coincide con lo reportado, el tiempo de ejecución y si la solución es óptima comparada con la de programación dinámica (y por cuánto difiere si no lo es). Con la columna de costos es fácil comprobar a mano el esfuerzo de una solución.
- **Guardar la salida** del algoritmo seleccionado en un archivo de texto con el formato pedido.

6.3. Formatos de entrada y salida

La entrada tiene $n + 2$ líneas: n; una línea `op,recep,inf` por agente; y $R_{\text{máx}}$. El lector acepta como separadores comas, punto y coma, espacios y "`. `" (punto seguido de espacio, como aparece en el enunciado), tolera líneas vacías y finales de línea de Windows, y valida los rangos ($o_i \in [-100, 100]$, $r_i \in [0, 1]$, $inf_i \in [1, 3]$, $R_{\text{máx}} \geq 0$), informando la línea del error. La salida tiene $n + 2$ líneas: Ext (con seis decimales), Esfuerzo, y un 0 o 1 por agente.

7. Comparación de las tres soluciones

7.1. Diseño de los casos de prueba

Los casos se diseñaron en cuatro grupos:

1. **Ejemplos del enunciado** (E_1, E_2), para validar el cálculo del esfuerzo y del extremismo contra valores conocidos.

2. **Casos borde** (E_5, E_6, E_7): $R_{\text{máx}} = 0$, presupuesto que alcanza para todos, agentes con receptividad 1 (costo 0) y opinión 0 (beneficio 0), y valores que exponen errores de redondeo.

3. **Casos adversarios para el voraz** (E_3, E_4, E_8), construidos a partir del análisis de la sección 4.4: presupuesto desperdiciado, agente muy extremo bloqueado por uno barato y óptimos empatados.

4. **Casos aleatorios**, generados con la misma función que usa la interfaz: o_i uniforme en $\{-100, \ldots, 100\}$, r_i uniforme en $[0, 1]$ e inf_i uniforme en $[1, 3]$ (con dos decimales), y $R_{\text{máx}}$ uniforme entre 0 y $\sum_i c_i$ para cubrir desde presupuestos muy escasos hasta suficientes. Se usó una semilla fija para que los resultados sean reproducibles.

Para cada caso se verifica automáticamente (programa `pruebas`) que las tres salidas sean aplicables, que el esfuerzo y el extremismo reportados coincidan con los recalculados a partir de E, y que fuerza bruta y programación dinámica den el mismo extremismo. Todas las verificaciones pasaron; en particular, **la programación dinámica coincidió con la fuerza bruta en los** 2000 **casos aleatorios con** $n \leq 14$ del programa de pruebas y en los 3200 casos con $n \leq 20$ de la tabla 7.

7.2. Resultados en los casos diseñados

		Fuerza bruta		Voraz		Dinámica	
Caso	Ext inicial	Esf	Ext	Esf	Ext	Esf	Ext
E_1	47.2582	50	33.4996	50	33.4996	50	33.4996
E_2	21.6025	28	9.4281	28	9.4281	28	9.4281
E_3	30.9121	10	20.0000	6	**23.5702**	10	20.0000
E_4	50.9902	50	10.0000	50	10.0000	50	10.0000
E_5	33.0246	0	17.0477	0	17.0477	0	17.0477
E_6	69.3462	328	0.0000	328	0.0000	328	0.0000
E_7	25.4951	3	25.0000	3	25.0000	3	25.0000
E_8	32.6803	75	24.4949	75	24.4949	75	24.4949

Tabla 6: Resultados de los tres algoritmos en los casos diseñados.

7.3. Calidad del voraz en casos aleatorios

Para $n \le 20$ el óptimo se calculó con fuerza bruta (y se comprobó que la dinámica da lo mismo); para $n = 100$ y $n = 1000$ se usó la dinámica. El error relativo es $(\mathrm{Ext}_V - \mathrm{Ext}^*)/\,\mathrm{Ext}^*$, y B_V/B^* es la fracción del beneficio óptimo que logra el voraz.

n	Casos	Voraz óptimo	Error prom. en Ext	Error máx. en Ext	B_V/B^* mín.
5	1000	88.4 %	2.47 %	206.2 %	0.729
10	1000	71.9 %	2.75 %	105.8 %	0.794
15	1000	61.6 %	2.24 %	147.2 %	0.778
20	200	56.0 %	1.97 %	60.2 %	0.888
100	200	27.5 %	0.21 %	3.09 %	0.982
1000	200	17.5 %	0.005 %	0.12 %	0.997

Tabla 7: Calidad del algoritmo voraz en redes aleatorias.

Se observa que:

- El voraz siempre logró al menos el 72.9 % del beneficio óptimo, por encima de la garantía teórica del 50 %.
- La probabilidad de obtener el óptimo exacto baja al crecer n, pero el error relativo cae rápidamente: para $n = 1000$ el extremismo del voraz difiere en promedio un 0.005 % del óptimo.
- Los errores máximos grandes en Ext aparecen en redes pequeñas donde el óptimo deja un extremismo cercano a 0, de modo que una diferencia absoluta pequeña es grande en términos relativos.

7.4. Tiempos de ejecución

Las mediciones se hicieron en un Intel Core i7-1165G7 a 2.8 GHz con compilación optimizada (`-O2`), con $R_{\text{máx}}$ igual a la mitad del esfuerzo de moderar a todos.

n	FB (ms)
4	0.002
8	0.035
12	0.746
16	12.9
18	61.0
20	296.7
22	1 203.3
24	4 403.7

n	A	Voraz (ms)	Dinámica (ms)
10	267	0.0006	0.005
20	554	0.0007	0.020
100	2 604	0.0034	0.417
500	11 874	0.018	8.61
1000	26 583	0.059	36.7
2000	52 536	0.142	130.2
5000	126 149	0.450	788.6

Tabla 8: Tiempos de ejecución medidos.

Los tiempos confirman el análisis: la fuerza bruta se multiplica por algo más de 2 con cada agente adicional ($\Theta(n \cdot 2^n)$); la dinámica crece aproximadamente con el cuadrado de n (al pasar de $n = 1000$ a $n = 2000$ el tiempo se multiplica por 3.6, y A crece linealmente con n); el voraz es casi lineal. Extrapolando, la fuerza bruta tardaría unos 6 minutos con $n = 30$, unos 6 días con $n = 40$ y unos 20 años con $n = 50$, mientras que la dinámica resuelve $n = 5000$ en menos de un segundo.

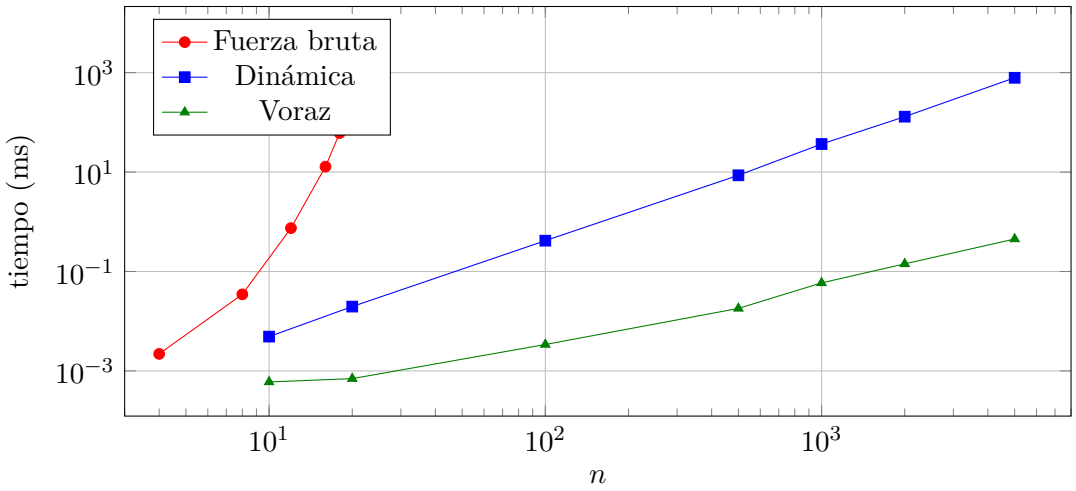


Figura 1: Tiempos de ejecución en escala logarítmica.

8. Conclusiones

	Fuerza bruta	Voraz	Dinámica
Tiempo	$\Theta(n \cdot 2^n)$	$\Theta(n \log n)$	$\Theta(n \cdot A) = O(n^2)$
Espacio	$\Theta(n)$	$\Theta(n)$	$\Theta(n \cdot A) = O(n^2)$
¿Óptimo?	Siempre	No siempre ($\geq 1/2$ de B^*)	Siempre
Tamaño práctico	$n \lesssim 25$	Millones de agentes	Miles de agentes

Tabla 9: Resumen de las tres estrategias.

- **Fuerza bruta.** Su ventaja es la simplicidad y la certeza: es fácil de implementar y de demostrar correcta, y sirve como referencia para validar los otros algoritmos (así se usó en las pruebas). Su desventaja es el crecimiento exponencial, que la hace inútil más allá de unos 25 agentes.
- **Voraz.** Es, por mucho, el más rápido, con un costo dominado por un ordenamiento, y usa memoria lineal. En la práctica sus soluciones son muy buenas, y con muchos agentes casi indistinguibles del óptimo. Su desventaja es que no garantiza el óptimo: existen entradas pequeñas donde se equivoca, y solo garantiza la mitad del beneficio óptimo. Es la mejor opción cuando la red es muy grande o se necesita una respuesta inmediata y se tolera un error pequeño.
- **Programación dinámica.** Garantiza el óptimo en tiempo polinomial gracias a que el presupuesto útil está acotado por $300n$, y resolvió miles de agentes en menos de un segundo. Su desventaja es la memoria: la tabla de decisiones crece como n^2 y es el primer recurso que se agota (alrededor de $n \approx 2^{10}$ a 2^{12} con 4 bytes por celda). Es la mejor opción cuando se necesita la solución exacta y la red tiene hasta algunos miles de agentes.
- Reconocer que ModEx es una mochila 0/1 fue clave: explica por qué el voraz por razón falla, de dónde sale su garantía, y cuál es la subestructura óptima que aprovecha la programación dinámica.